PAMANTASAN NG LUNGSOD NG PASIG (PLP)

Document Management System (DMS) Project Report

Background • Project Objectives • Outputs • Limitations
Task Board • Demonstration Checklist • Presentation Arrangements

Document	DMS Project Report (evaluator submission draft)
Date	28 October 2026
Status	Draft — pending evaluator confirmation of date, document inventory and target users
Application	PLP Document Management System — Laravel 12, PHP 8.2, MySQL (InnoDB)
Prepared from	README.md, docs/AGENTS.md (SDD authority), docs/FRONTEND.md, docs/DMS-SDD.pdf, docs/API.md
Data handling	Synthetic / redacted data only. No real passwords, connector tokens, or student/employee records are disclosed.

1. Background

School offices at Pamantasan ng Lungsod ng Pasig (PLP) need a controlled place to store supporting files, preserve earlier versions of those files, and share authorized copies across departments. Each of the school's four enterprise resource planning (ERP) systems remains responsible for its own business records and processes; the Document Management System (DMS) is a separate, centralized document repository, not a replacement for any ERP's existing record screens.

The DMS receives documents and transaction history from four ERP systems:

Code	Connected system	DMS responsibility
EMS	Employee Management	Receive and organize approved personnel-related document categories and source history.
PMS	Payroll Management	Receive and organize approved payroll-related supporting files and source history.
ES	Enrollment	Receive and organize approved enrollment-related supporting files and source history.
OAS	Online Admission	Receive and organize approved admission-related supporting files and source history.

The DMS is a single standalone Laravel 12 application (PHP 8.2) with one MySQL (InnoDB) database and one private file store. There are no microservices and no managed cloud database, queue, search, or object storage. Only the `public/` directory is web-served; all version files live in private storage under `storage/app/private/documents/<version UUID>`.

Key scope boundaries that shape the background:

- The DMS does **not** recalculate payroll, decide admissions, process enrollment, or modify source records in any ERP.
- Students are never DMS users; the system serves staff and administrative roles only.
- All ERP integration goes through `/api/v1` only, with scoped bearer tokens and idempotent receipts; there is no cross-database access between ERP systems.
- Every sensitive action (upload, download, modify, delete, access change) is written to the audit log.
- Files are validated by actual content, limited to 10 MB, SHA-256 checksummed, and scanned with a local ClamAV engine before being served.
- File versions are immutable: earlier versions are never overwritten or deleted (except soft-trash of the parent document).

> **Compliance context (to be confirmed):** The design references the Philippine Data Privacy Act of 2012 (RA 10173) and requires a DPIA before go-live; children's personal data in ES/OAS payloads is restricted to office and admin roles. Hosting model, data residency, audit retention, SSO/MFA, encryption at rest, monitoring and penetration testing are open items (TBD) and are not certified by code.

2. Project Objectives

The project objectives, as recorded in the working checklist (README §1), are:

1. **Store documents privately and preserve immutable file versions.** Files are stored privately with UUID paths, detected MIME metadata, SHA-256 integrity checks, and queued ClamAV scanning; versions are never overwritten.
2. **Give authorized users the correct viewing, editing and management access.** A three-role hierarchy (Super Admin, Admin, Staff) is enforced server-side by Laravel policies/Gates and route middleware, scoped by school, campus and ERP source.
3. **Support justified document sharing across offices without unnecessary copies.** Named-user document and folder grants (Viewer, Commenter, Editor) with optional expiry, revocation, pinned versions and folder inheritance; named shares may cross ERP source boundaries as a documented exception.
4. **Keep sensitive actions traceable through audit records.** Audit entries record actor, action, document, school/campus and revision; Super Admin has full scoped audit, Admins have source-scoped audit.
5. **Provide standalone document workflows and a scoped integration contract for the four ERPs.** The DMS works without connected ERPs, and exposes `/api/v1` document/history endpoints with revision handling and replay protection.
6. **Demonstrate functional jQuery/jQuery UI features and responsive Bootstrap interfaces.** The frontend uses Blade, Bootstrap 5.3.3, jQuery 3.7.1 and jQuery UI 1.14.1, including draggable documents, a four-second notification toast with progress bar, and role-aware menus.

Underlying SDD objectives include low running cost (OBJ-04) — no paid infrastructure without an explicit architecture decision — and defined availability/recovery targets (99.5% monthly availability, RPO 24 h / RTO 8 h, daily encrypted coordinated backups, quarterly recovery drills, three-year minimum audit retention with legal holds) that require external evidence and are not certified by code.

3. Outputs

This section summarizes the implemented screens and workflows, the evidence (screenshot/code mapping) available for the report, current test status, and the remaining limitations.

3.1 Implemented screens and workflows

Area	Current implementation	Evidence to explain (code / screens)
Accounts & roles	Super Admin, Admin and Staff. Laravel policies/Gates plus route middleware enforce the server-side role matrix; authorization is enforced in code, not by hiding controls.	`app/Policies`, `app/Providers/AppServiceProvider.php`, `routes/web.php`, `resources/views/layout.blade.php`
Source oversight	Super Admin reads every ERP source and its audit log without editing; Admin manages one assigned source; Staff reads its own source. Super Admin document/folder management is read-only.	Role dashboards/navigation; admin views; audit screen
Uploads	Plain multi-file drag/drop or file selection; filename titles; selectable classification; two simultaneous requests; retry failed files. Up to 50 files per upload session and 10 MB per file; any extension accepted, scan results determine whether it can be served.	`resources/views/partials/upload.blade.php`, `public/js/dms.js`, `app/Http/Requests/UploadRequest.php`
File security	Private UUID storage, detected MIME metadata, SHA-256 integrity checks, queued local ClamAV scanning. Failed, pending and quarantined versions remain blocked; only Clean versions are served after a fresh SHA-256 check.	`app/Services/DocumentService.php`, ClamAV scan-retry Artisan command

Preview	Local PDF.js viewer; common raster images; escaped text, CSV and JSON previews. Office documents and archives remain download-only; no external document viewer receives private files.	`resources/views/document.blade.php`, local PDF.js
Organization	Folders, rename, classification editing, move menu, document-to-folder drag/drop (jQuery UI), stars, trash/restore, Back navigation.	`public/js/dms.js`, `resources/views/partials/documents.blade.php`, `app/Http/Controllers/DocumentController.php`
Sharing	Named-user document and folder grants: Viewer, Commenter or Editor; optional expiry, revocation, pinned versions, folder inheritance. Folder grants expose Internal documents only; only Super Admin can create or revoke a Restricted share. No public/anonymous links.	`app/Services/Access.php`, `app/Http/Controllers/ShareController.php`
Integration	Scoped connector tokens and `/api/v1` document/history endpoints; revision handling and replay protection (idempotency: identical replay → 200 with original receipt; changed hash / stale revision → 409). API read-back exposes Internal and Confidential documents only, never Restricted.	`app/Http/Controllers/HistoryController.php`, `docs/API.md`, `docs/openapi.json`
Duplicate handling	SHA-256 is recorded; idempotency keys prevent duplicate processing of the same submission retry.	UploadRequest + history receipt logic

Frontend	Blade, Bootstrap 5.3.3, jQuery 3.7.1, jQuery UI 1.14.1; role-aware menus; four-second top-right notifications with progress bar (jQuery UI widget).	`resources/views/workspace.blade.php`, `resources/views/admin.blade.php`, `public/css/dms.css`, `public/js/dms.js`

3.2 Evidence structure for the report

The report follows the professor's evidence set (README §4), pairing each demonstration with code to explain and a rationale. Synthetic/redacted data only.

Screenshot / demonstration	Code to explain	Rationale
Different role dashboards/navigation	`resources/views/layout.blade.php`, `app/Policies`, `routes/web.php`	Different controls plus policy/Gate enforcement; hidden controls alone are insufficient.
Multi-file upload and classification	`resources/views/partials/upload.blade.php`, `public/js/dms.js`, `app/Http/Requests/UploadRequest.php`	jQuery queues separate uploads; validated metadata and automatic scanning.
Drag document into folder; move menu	`public/js/dms.js`, `resources/views/partials/documents.blade.php`, `app/Http/Controllers/DocumentController.php`	jQuery UI interaction with the same validated, revision-checked server operation.
Four-second top-right notification	`public/js/dms.js`, `resources/views/layout.blade.php`	jQuery UI progress bar provides visible feedback and dismissal timing.
Sharing and a denied action	`app/Services/Access.php`, `app/Http/Controllers/ShareController.php`	Named-user grants, role limits, scope and version restrictions.
Phone/tablet/desktop tables and dialogs	`resources/views/workspace.blade.php`, `resources/views/admin.blade.php`, `public/css/dms.css`	Bootstrap layout/table use and tested responsive behavior.

Preview, classification change, version history	`resources/views/document.blade.php`, `app/Services/DocumentService.php`	Inspect content without downloading; access/checksum/scan gates still apply.
Folder/group collaboration and duplicate detection	Final implementation, once completed	Present as backlog/design work until functional and tested.

> **Short code example (jQuery UI notification countdown):**
> `const countdown = $(box).find(".toast-countdown").progressbar({ max: 4000, value: 4000 });`
> The timer updates the progress value and dismisses the toast after four seconds — a working jQuery UI widget, not merely an included script.

3.3 Suggested demonstration sequence

1. Explain the confirmed document use case and its difference from the originating ERP.
2. Sign in as Staff; upload sample files, select classification and preview a clean document.
3. Demonstrate a new version, authorized editing, search and moving an owned file.
4. Sign in as Admin; show source-scoped staff/content management and named-user sharing.
5. Use recipient accounts to demonstrate allowed viewing/editing and a blocked action.
6. Sign in as Super Admin; show system administration, read-only ERP oversight, full audit and account management.
7. Show folder collaboration and duplicate detection only once implemented; otherwise identify the exact remaining work.
8. Show mobile/tablet/desktop screenshots and explain the actual Bootstrap/jQuery code.

3.4 Test results and verification status

- PHPUnit tests run against an isolated in-memory SQLite database by default (`php artisan test`).
- Frontend: `npm test`, `npm run build`, `npm run format:check`, `php scripts/check-encoding.php`.
- MySQL concurrency, restore and real-engine ClamAV checks have separate environment requirements; skipped checks are recorded rather than claimed as passed.
- GitHub Actions runs the same checks on pushes/pull requests, plus a post-merge smoke test and a scheduled real-ClamAV workflow (see docs/CI-CD.md).
- Responsive visual audits and screenshot evidence are maintained under `docs/ui-evidence/`.

- **Caveat:** this report documents current behavior; checked items describe existing functionality, not final evaluator acceptance. No overall completion percentage is claimed.

4. Limitations

The following limitations are acknowledged and must not be presented as completed work:

- **Department/group permissions:** not implemented. Folder grants cover named accounts only and expose Internal documents; college/department group membership (e.g. a CCS faculty group) is not modeled.
- **Duplicate-file detection:** detecting the same file in separate uploads with different idempotency keys is backlog work. The warning/block/offer-existing/new-version behavior has not been decided.
- **Previews:** office documents and archives are download-only. The agreed file-preview format list is TBD (client).
- **Scanner/worker health:** deployed ClamAV scanner and background worker health still needs verification.
- **Zebra tables:** consistent Bootstrap striped/zebra table styling with alternating gray/white rows is not yet complete; current `.table` usage alone does not finish the requirement.
- **Responsive coverage:** the phone/tablet/desktop checklist, portrait/landscape checks and keyboard-navigation checks are not yet complete; final screenshots are captured only after these checks.
- **Document inventory:** candidate document types (student grade document, system review/evaluation, other office document) are pending confirmation with the professor and ERP teams.
- **Target users:** audiences for individual faculty, department heads, deans, college secretaries and other school staff still need decisions; source-system assignment and college/department membership are not yet finalized.
- **Sharing rationale:** one real cross-department sharing scenario and the reason each recipient needs access is pending confirmation.
- **ERP adapters:** each ERP team must supply and test its own adapter; working endpoints do not mean all four real ERPs are connected.
- **Out of scope (do not build):** student accounts/access, external/anonymous public sharing links, folder-level sharing (new), in-browser document editing, business approval of admissions/enrollment/employment/payroll, direct modification of ERP records or databases, source-side adapters/outbox (owned by ERP teams), SSO/MFA, encryption at rest, central monitoring, penetration testing.

- **Compliance not certified by code:** 99.5% availability, RPO/RTO, backups, recovery drills, audit retention, DPIA, data residency, hosting model and client sign-offs require external evidence.

> **Unverified items rule:** do not mark a decision task complete without the agreed answer, and do not mark an implementation task complete without a working demonstration/check. TBDs are flagged, not invented.

5. Prioritized Work Board

Priority	Action	Completion evidence
P0	Confirm document inventory, target users and the reason for cross-department sharing.	Reviewed inventory and agreed user journeys.
P0	Agree organizational/account scope and classification access policy.	Permission matrix covering roles, colleges/departments and file sensitivity.
P0	Specify folder/group collaboration and duplicate behavior.	Approved design with inheritance, revocation, move and privacy rules.
P1	Implement and test the approved gaps.	Working folder collaboration and duplicate-detection scenarios with regression checks.
P1	Complete zebra tables and responsive/role UI checks.	Phone/tablet/desktop evidence and denied-action demonstrations.
P1	Write the report and capture final screenshot/code pairs.	Background, objectives, output and limitations sections (this document).
P2	Rehearse the presentation and coordinate integration demonstrations.	Tested demo accounts, sample files and confirmed evaluator arrangements.

Owners and estimates have not been assigned. Priorities are ordered P0 → P1 → P2; do not start P2 while P1 report work remains open.

6. Demonstration Checklist

✓	Item	Status
☐	Confirm one real cross-department sharing scenario and the reason each recipient needs access.	Pending professor confirmation
☐	Prepare synthetic/sample documents and accounts for the demonstration.	To do
☐	Demonstrate Viewer, Commenter and Editor as separate signed-in users.	To do
☐	Show a denied action as well as a successful action.	To do

	Demonstrate revocation/expiry and pinned-version restrictions.	To do
	Explain school/campus boundaries and how proposed college/department groups would fit them.	Pending scope decision
	Demonstrate Super Admin, Admin and Staff interfaces separately.	To do
	Prove Staff cannot access administration URLs or unshared documents through direct requests (403/404 enforced server-side).	Implemented; needs demo
	Show mobile/tablet/desktop screenshots and explain Bootstrap/jQuery code.	Responsive checks incomplete
	Present folder collaboration and duplicate detection only once implemented; otherwise identify exact remaining work.	Backlog

7. Presentation Arrangements

7.1 Schedule

Date	Meeting note	Status
24 October 2026	Evaluator unavailable.	Availability noted; do not schedule without reconfirming.
31 October 2026	Preferred fallback presentation date.	Tentative; confirm with evaluator.
7 November 2026	Alternative presentation date.	Tentative; confirm with evaluator.

7.2 Arrangements to confirm

- Final date/time and online versus face-to-face modality.
- Whether frontend and backend evaluations are presented separately.
- Assigned presenters, demo roles and report responsibilities.
- A local demo and recorded/screenshot evidence in case the live environment is unavailable.

These notes are not a confirmed booking or an automated reminder.

7.3 Demo accounts and sample files

- Use synthetic/redacted sample documents and accounts only.
- Separate signed-in accounts for Super Admin, Admin (assigned ERP source), Staff, and sharing recipients (Viewer/Commenter/Editor).
- Do not expose passwords, connector tokens, or real student/employee records in the demo or screenshots.

7.4 Fallback evidence

- Recorded walkthrough of the demonstration sequence (README §4).
- Final screenshot/code pairs for each evidence item (README §4 table).
- Responsive screenshots at phone/tablet/desktop sizes once checks are complete.

Source documents and data-handling statement

This report was assembled from the repository's own planning documents: `README.md` (working checklist, professor feedback, report deliverables, task board, presentation schedule), `docs/AGENTS.md` (latest SDD implementation authority, roles, constraints, out-of-scope and TBD items), `docs/FRONTEND.md` (frontend stack and live testing), `docs/DMS-SDD.pdf`, `docs/API.md` and `docs/openapi.json`.

All data used in the report, demonstration and screenshots is synthetic or redacted. No real passwords, connector tokens, or student/employee records are disclosed. Checked items describe existing functionality, not final evaluator acceptance; unchecked items need decisions, implementation or presentation evidence.